

Fluid-structure interaction

Giovanni Carpenedo, Simone Ferri, Emma Scalvedi

A. Y. 2025/2026

1 Introduction

Fluid-Structure Interaction (FSI) represents a class of multiphysics problems in computational mechanics. It describes the coupling between the behavior of a fluid and a solid structure, and therefore the connection between laws of fluid dynamics and laws of elasticity.

FSI is recurring in many real-world applications, such as the flow of blood through elastic arteries or the wind-induced vibration of bridges, in which the interaction is bidirectional: the fluid exerts a force on the solid causing its deformation, while the latter alters the fluid flow.

This project aims to model a 2D steady linear FSI problem using a monolithic solver approach based on deal.II and Trilinos with MPI parallelization.

1.1 The physical and mathematical model

The problem is defined on a global domain Ω , given by the union of the fluid subdomain Ω_f and the solid subdomain Ω_s , in figure 1. The intersection $\Sigma = \partial\Omega_f \cap \partial\Omega_s$ represents the physical interface where the exchange of momentum and energy occurs.

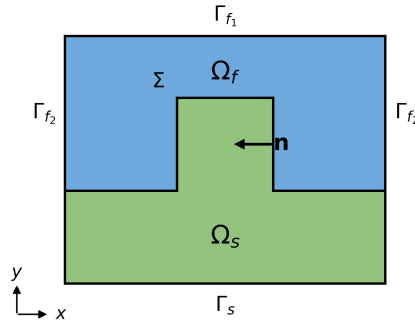


Figure 1: The global domain Ω .

1.1.1 Stokes equations

In Ω_f , we model the fluid as an incompressible flow under the assumption of a very low Reynolds number ($\text{Re} \ll 1$). This is described by the Stokes equations; the first one,

$$-\nu \Delta \mathbf{u} + \nabla p = \mathbf{0} \quad (1)$$

expresses momentum balance, and states that, for a steady flow with no external forces, the internal viscous forces (represented by the Laplacian of velocity $\Delta \mathbf{u}$ scaled by viscosity ν) are exactly balanced by the pressure gradient ∇p . The second equation,

$$\nabla \cdot \mathbf{u} = 0 \quad (2)$$

comes from the mass conservation equation $\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$. The incompressibility constraint ensures that the volume of any fluid element does not change over time and therefore density ρ is constant. We obtain that the velocity field must be divergence-free.

1.1.2 Linear elasticity

In Ω_s , the structure is governed by the equations of linear elasticity, which assume small displacements and linear material behavior:

$$-\nabla \cdot \sigma(\mathbf{d}) = \mathbf{0} \quad (3)$$

This means mechanical equilibrium: internal stresses within the solid balance everywhere, so there is no net force on any material element.

The stress tensor $\sigma(\mathbf{d})$ is linked to the displacement $\mathbf{d}$ via the constitutive equation of linear isotropic elasticity $\sigma(\mathbf{d}) = 2\mu \boldsymbol{\varepsilon} + \lambda (\nabla \cdot \mathbf{d}) \mathbf{I}$, where λ and μ are the Lamé parameters, which characterize the material's stiffness and resistance to shear, while $\boldsymbol{\varepsilon} = \frac{1}{2} (\nabla \mathbf{d} + \nabla \mathbf{d}^T)$ is the strain tensor.

1.1.3 Material uniformity

The material properties (ν , λ , μ) are assumed to be constant within their respective subdomains.

1.1.4 Coupling and interface conditions

The interaction at the interface Σ is governed by two physical principles: kinematic continuity and dynamic equilibrium.

Under the assumption of a steady-state regime and small displacements, the solid structure reaches a static equilibrium, meaning it has no velocity at its boundary. Consequently, the kinematic continuity at the interface enforces that the fluid velocity matches the stationary solid, effectively reducing to the classic no-slip condition ($\mathbf{u} = \mathbf{0}$).

The equation $\sigma(\mathbf{d})\mathbf{n} = \nu \nabla \mathbf{u} \mathbf{n} - p \mathbf{n}$ ensures Newton's Third Law: "For every action, there is an equal and opposite reaction". In particular, the force per unit area exerted by the fluid's viscous stress and pressure onto the solid is exactly matched by the solid's internal stress response.

1.2 Monolithic solver

In FSI problems, two types of solver can be chosen: partitioned and monolithic. We have implemented a monolithic solver, where the fluid and solid equations are assembled into a single algebraic system and solved simultaneously.

In general FSI applications, the primary motivation for this choice is stability. Partitioned solvers often suffer from “artificial added mass effects”, a numerical instability that occurs when fluid and solid densities are similar. Because computations are disconnected, the algorithm overestimates the fluid’s inertia, creating an escalating loop of errors, which is particularly problematic in incompressible flows.

Although our specific mathematical model assumes small displacements and a steady-state regime (effectively resulting in a one-way coupled system where the solid deformation does not alter the fluid domain), employing a monolithic scheme remains structurally advantageous. It builds a robust foundational framework that strictly enforces kinematic and dynamic coupling conditions at the interface. This exact satisfaction of equilibrium at the algebraic level ensures numerical stability and creates an architecture that is intrinsically prepared for highly deformable, fully coupled, or transient extensions, bypassing the limitations of partitioned approaches.

1.3 Boundary conditions

We impose a non-homogeneous Dirichlet condition on the top boundary $\Gamma_{f1} \subset \partial\Omega \cap \partial\Omega_f$, where the velocity follows a sinusoidal profile: $\mathbf{u} = (0, \sin(\pi x))^T$ in 2D, like in Figure 2, and $\mathbf{u} = (0, 0, \sin(\pi x) \sin(\pi y))^T$ in 3D.

On the remaining fluid boundaries, denoted as Γ_{f2} , a free-flow (or outflow) condition is applied. This is modeled as a homogeneous Neumann boundary condition, $\nu \nabla \mathbf{u} \mathbf{n} - p \mathbf{n} = 0$, which ensures that the fluid is free to leave the domain without experiencing any external normal stress.

The solid is fixed along its outer perimeter $\Gamma_s = \partial\Omega \cap \partial\Omega_s$, enforcing $\mathbf{d} = \mathbf{0}$.

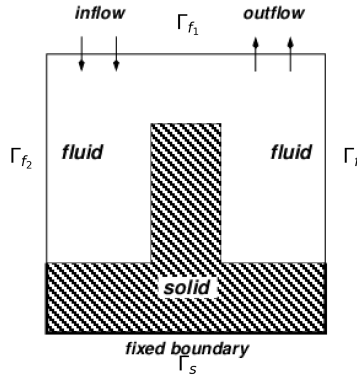


Figure 2: Domain with Dirichlet condition.

2 From the strong formulation to the finite element formulation

2.1 Weak formulation

Given $\Omega = \Omega_f \cup \Omega_s$, where Ω_f is the region of the domain occupied by the fluid and Ω_s is the one occupied by the solid, $\Sigma = \partial\Omega_f \cap \partial\Omega_s$ is the fluid-solid interface, we state the strong formulation of the problem as:

$$\left\{ \begin{array}{ll} -\nu\Delta \mathbf{u} + \nabla p = \mathbf{0} & \text{in } \Omega_f \\ \nabla \cdot \mathbf{u} = 0 & \text{in } \Omega_f \\ -\nabla \cdot \sigma(\mathbf{d}) = 0 & \text{in } \Omega_s \\ \mathbf{u} = \mathbf{0} & \text{on } \Sigma \\ \sigma(\mathbf{d})\mathbf{n} = \nu\nabla \mathbf{u}\mathbf{n} - p\mathbf{n} & \text{on } \Sigma \\ \mathbf{u} = \mathbf{u}_0 & \text{on } \Gamma_{f1} \\ \nu\nabla \mathbf{u}\mathbf{n} - p\mathbf{n} = 0 & \text{on } \Gamma_{f2} \\ \mathbf{d} = 0 & \text{on } \Gamma_s \end{array} \right. \quad (4)$$

where $\Gamma_{f1} \subset \partial\Omega \cap \partial\Omega_f$, $\Gamma_{f2} = (\partial\Omega \cap \partial\Omega_f) \setminus \Gamma_{f1}$ and $\Gamma_s = \partial\Omega \cap \partial\Omega_s$, as shown in Figure 1.

We want to derive the weak formulation of the problem by multiplying each equation by a test function, integrating the results on the domain, and then summing together the integrals, since they are all equal to zero.

For the first equation we multiply by $\mathbf{v} \in V_0$ and integrate over Ω_f , obtaining:

$$\int_{\Omega_f} (-\nu\Delta \mathbf{u} + \nabla p) \cdot \mathbf{v} dx = 0 \quad (5)$$

We apply the Green formula and the result is

$$\int_{\Omega_f} \nu \nabla \mathbf{u} : \nabla \mathbf{v} dx - \int_{\Omega_f} p \nabla \cdot \mathbf{v} dx - \int_{\partial\Omega_f} (\nu \nabla \mathbf{u}\mathbf{n}_f - p\mathbf{n}_f) \cdot \mathbf{v} ds = 0 \quad (6)$$

where $\mathbf{n}_f$ is the normal unit vector going from the fluid to the solid, so $\mathbf{n}_f = \mathbf{n}$.

Knowing that $\partial\Omega_f = \Sigma \cup \Gamma_{f1} \cup \Gamma_{f2}$, we can split the integral on $\partial\Omega_f$, obtaining

$$\int_{\Sigma} (\nu \nabla \mathbf{u}\mathbf{n}_f - p\mathbf{n}_f) \cdot \mathbf{v} ds + \int_{\Gamma_{f1}} (\nu \nabla \mathbf{u}\mathbf{n}_f - p\mathbf{n}_f) \cdot \mathbf{v} ds + \int_{\Gamma_{f2}} (\nu \nabla \mathbf{u}\mathbf{n}_f - p\mathbf{n}_f) \cdot \mathbf{v} ds \quad (7)$$

where

$$\int_{\Sigma} (\nu \nabla \mathbf{u}\mathbf{n}_f - p\mathbf{n}_f) \cdot \mathbf{v} ds = 0 \quad (8)$$

and

$$\int_{\Gamma_{f1}} (\nu \nabla \mathbf{u}\mathbf{n}_f - p\mathbf{n}_f) \cdot \mathbf{v} ds = 0 \quad (9)$$

because we impose $\mathbf{v} = \mathbf{0}$ on Σ and Γ_{f1} , and

$$\int_{\Gamma_{f2}} (\nu \nabla \mathbf{u} \mathbf{n}_f - p \mathbf{n}_f) \cdot \mathbf{v} ds = 0 \quad (10)$$

due to the free-flow condition. Consequently, from the first equation we obtain:

$$\int_{\Omega_f} \nu \nabla \mathbf{u} : \nabla \mathbf{v} dx - \int_{\Omega_f} p \nabla \cdot \mathbf{v} dx = 0 \quad (11)$$

For the second equation we choose $q \in Q$ as test function, multiplying and integrating on Ω_f we obtain

$$\int_{\Omega_f} \nabla \cdot \mathbf{u} q dx = 0 \quad (12)$$

For the third equation we choose $\mathbf{w} \in W$ as test function, multiplying and integrating on Ω_s we obtain

$$\int_{\Omega_s} -\nabla \cdot \sigma(\mathbf{d}) \mathbf{w} dx = 0 \quad (13)$$

Applying the Green formula, we have

$$\int_{\Omega_s} \sigma(\mathbf{d}) : \epsilon(\mathbf{w}) dx - \int_{\partial\Omega_s} (\sigma(\mathbf{d}) \mathbf{n}_s) \cdot \mathbf{w} ds = 0 \quad (14)$$

where $\mathbf{n}_s$ is the normal unit vector going from the solid to the fluid, so $\mathbf{n}_s = -\mathbf{n}$ and $\epsilon(\mathbf{w}) = \frac{1}{2}(\nabla \mathbf{w} + (\nabla \mathbf{w})^T)$ is the symmetric gradient.

The gradient $\nabla \mathbf{w}$ can be decomposed in its symmetric part $\epsilon(\mathbf{w})$ and its antisymmetric part $\omega(\mathbf{w})$, so

$$\sigma(\mathbf{d}) : \nabla \mathbf{w} = \sigma(\mathbf{d}) : \epsilon(\mathbf{w}) + \sigma(\mathbf{d}) : \omega(\mathbf{w}) \quad (15)$$

We notice that $\sigma(\mathbf{d}) : \omega(\mathbf{w})$ is the product between a symmetric and an antisymmetric tensor, so it is equal to zero; for this reason $\sigma(\mathbf{d}) : \nabla \mathbf{w} = \sigma(\mathbf{d}) : \epsilon(\mathbf{w})$.

We can split the integral on $\partial\Omega_s$ in $\int_{\Sigma} (\sigma(\mathbf{d}) \mathbf{n}) \cdot \mathbf{w} ds$ and $\int_{\Gamma_s} (\sigma(\mathbf{d}) \mathbf{n}) \cdot \mathbf{w} ds$ where

$$\int_{\Gamma_s} (\sigma(\mathbf{d}) \mathbf{n}) \cdot \mathbf{w} ds = 0 \quad (16)$$

because we impose $\mathbf{w} = \mathbf{0}$ on Γ_s , and

$$\int_{\Sigma} (\sigma(\mathbf{d}) \mathbf{n}) \cdot \mathbf{w} ds = \int_{\Sigma} (\nu \nabla \mathbf{u} \mathbf{n} - p \mathbf{n}) \cdot \mathbf{w} ds \text{ for the boundary conditions.} \quad (17)$$

Summing all contributions together:

$$\begin{aligned} & \int_{\Omega_f} \nu \nabla \mathbf{u} : \nabla \mathbf{v} dx - \int_{\Omega_f} p \nabla \cdot \mathbf{v} dx + \int_{\Omega_f} \nabla \cdot \mathbf{u} q dx + \\ & + \int_{\Omega_s} \sigma(\mathbf{d}) : \epsilon(\mathbf{w}) dx + \int_{\Sigma} (\nu \nabla \mathbf{u} \mathbf{n} - p \mathbf{n}) \cdot \mathbf{w} ds = 0 \end{aligned} \quad (18)$$

Then, splitting the integral on Σ , we obtain

$$\begin{aligned} & \int_{\Omega_f} \nu \nabla \mathbf{u} : \nabla \mathbf{v} dx - \int_{\Omega_f} p \nabla \cdot \mathbf{v} dx + \int_{\Omega_f} \nabla \cdot \mathbf{u} q dx + \\ & + \int_{\Omega_s} \sigma(\mathbf{d}) : \epsilon(\mathbf{w}) dx + \int_{\Sigma} (\nu \nabla \mathbf{u} \mathbf{n}) \cdot \mathbf{w} ds - \int_{\Sigma} (p \mathbf{n}) \cdot \mathbf{w} ds = 0 \end{aligned} \quad (19)$$

The weak formulation is well defined when the integrals are well defined; for this reason we impose:

$$\begin{aligned} \mathbf{u} & \in V = \{\mathbf{v} \in [H^1(\Omega_f)]^2 : \mathbf{v}|_{\Sigma} = 0, \mathbf{v}|_{\Gamma_{f1}} = \mathbf{u}_0\} \\ \mathbf{v} & \in V_0 = \{\mathbf{v} \in [H^1(\Omega_f)]^2 : \mathbf{v}|_{\Sigma} = 0, \mathbf{v}|_{\Gamma_{f1}} = 0\} \\ p, q & \in Q = \{q \in L^2(\Omega_f)\} \\ \mathbf{d}, \mathbf{w} & \in W = \{\mathbf{w} \in [H^1(\Omega_s)]^2 : \mathbf{w}|_{\Gamma_s} = 0\} \end{aligned} \quad (20)$$

The problem can be equivalently formulated in a setting where the Lax-Milgram lemma is applicable: in this case, the velocity function $\mathbf{u}$ and the test functions $\mathbf{v}$ must belong to the same space. For this reason, we introduce a lifting function $R\mathbf{u}_0 \in V$, such that $R\mathbf{u}_0|_{\Gamma_{f1}} = \mathbf{u}_0$; we can now define the function

$$\tilde{\mathbf{u}} \equiv \mathbf{u} - R\mathbf{u}_0 \in V_0 \quad (21)$$

Since the function $\tilde{\mathbf{u}}$ and the test functions $\mathbf{v}$ belong to the same functional space, the weak formulation can be framed within the context of Lax-Milgram lemma. We write $\mathbf{u}$ as $\tilde{\mathbf{u}} + R\mathbf{u}_0$ and we substitute it in (19), obtaining:

$$\begin{aligned} & \int_{\Omega_f} \nu \nabla \tilde{\mathbf{u}} : \nabla \mathbf{v} dx - \int_{\Omega_f} p \nabla \cdot \mathbf{v} dx + \int_{\Omega_f} \nabla \cdot \tilde{\mathbf{u}} q dx + \int_{\Omega_s} \sigma(\mathbf{d}) : \epsilon(\mathbf{w}) dx + \\ & + \int_{\Sigma} (\nu \nabla \tilde{\mathbf{u}} \mathbf{n}) \cdot \mathbf{w} ds - \int_{\Sigma} (p \mathbf{n}) \cdot \mathbf{w} ds = - \int_{\Omega_f} \nu \nabla (R\mathbf{u}_0) : \nabla \mathbf{v} dx \end{aligned} \quad (22)$$

We introduce the shorthand notation

$$\begin{aligned}
a_f : V \times V_0 &\rightarrow \mathbb{R}, & a_f(\mathbf{u}, \mathbf{v}) &= \int_{\Omega_f} \nu \nabla \mathbf{u} : \nabla \mathbf{v} dx \\
b : V \times Q &\rightarrow \mathbb{R}, & b(\mathbf{u}, q) &= - \int_{\Omega_f} \nabla \cdot \mathbf{u} q dx \\
a_s : W \times W &\rightarrow \mathbb{R}, & a_s(\mathbf{d}, \mathbf{w}) &= \int_{\Omega_s} \sigma(\mathbf{d}) : \epsilon(\mathbf{w}) dx \\
c_u : V \times W &\rightarrow \mathbb{R}, & c_u(\mathbf{u}, \mathbf{w}) &= \int_{\Sigma} (\nu \nabla \mathbf{u} \mathbf{n}) \cdot \mathbf{w} ds \\
c_p : Q \times W &\rightarrow \mathbb{R}, & c_p(p, \mathbf{w}) &= - \int_{\Sigma} (p \mathbf{n}) \cdot \mathbf{w} ds
\end{aligned} \tag{23}$$

Then, the weak formulation is:

$$\begin{aligned}
&\text{Find } \mathbf{u} \in V_0, p \in Q, \mathbf{d} \in W \text{ such that} \\
&a_f(\mathbf{u}, \mathbf{v}) + b^T(p, \mathbf{v}) + b(\mathbf{u}, q) + a_s(\mathbf{d}, \mathbf{w}) + c_u(\mathbf{u}, \mathbf{w}) + c_p(p, \mathbf{w}) = G(R\mathbf{u}_0, \mathbf{v}) \\
&\text{for all } \mathbf{v} \in V_0, q \in Q, \mathbf{w} \in W.
\end{aligned} \tag{24}$$

where $G(R\mathbf{u}_0, \mathbf{v}) = -a_f(R\mathbf{u}_0, \mathbf{v})$.

Since we want to solve the problem monolithically but $\mathbf{u}$ and p are defined only in the fluid subdomain while $\mathbf{d}$ in the solid one, we extend them by 0 to the respective subdomains in which they are not defined. So we introduce three new appropriate function spaces:

$$\begin{aligned}
\tilde{V} &= \{\tilde{\mathbf{v}}|_{\Omega_f} \in V_0, \tilde{\mathbf{v}}|_{\Omega_s} = 0\} \\
\tilde{Q} &= \{\tilde{q}|_{\Omega_f} \in Q, \tilde{q}|_{\Omega_s} = 0\} \\
\tilde{W} &= \{\tilde{\mathbf{w}}|_{\Omega_s} \in W, \tilde{\mathbf{w}}|_{\Omega_f} = 0\}
\end{aligned} \tag{25}$$

The new weak formulation is:

$$\begin{aligned}
&\text{Find } \mathbf{u} \in \tilde{V}, p \in \tilde{Q}, \mathbf{d} \in \tilde{W} \text{ such that} \\
&a_f(\mathbf{u}, \mathbf{v}) + b^T(p, \mathbf{v}) + b(\mathbf{u}, q) + a_s(\mathbf{d}, \mathbf{w}) + c_u(\mathbf{u}, \mathbf{w}) + c_p(p, \mathbf{w}) = G(R\mathbf{u}_0, \mathbf{v}) \\
&\text{for all } \mathbf{v} \in \tilde{V}, q \in \tilde{Q}, \mathbf{w} \in \tilde{W}.
\end{aligned} \tag{26}$$

2.2 Galerkin formulation

Let us introduce a uniform mesh of Ω into 2D discrete elements. We use the following subspaces defined on the triangulation T :

$$\begin{aligned}
V_h &= \{\mathbf{v}_h : \forall K \in T, \mathbf{v}_h|_K \in Q_{p+1}^2 \text{ if } K \subset \Omega_f, \mathbf{v}_h \in \mathcal{C}^0, \mathbf{v}_h|_K = 0 \text{ if } K \subset \Omega_s\} \\
Q_h &= \{q_h : \forall K \in T, q_h|_K \in Q_p \text{ if } K \subset \Omega_f, q_h \in \mathcal{C}^0, q_h|_K = 0 \text{ if } K \subset \Omega_s\} \\
W_h &= \{\mathbf{w}_h : \forall K \in T, \mathbf{w}_h|_K \in Q_r^2 \text{ if } K \subset \Omega_s, \mathbf{w}_h \in \mathcal{C}^0, \mathbf{w}_h|_K = 0 \text{ if } K \subset \Omega_f\}
\end{aligned} \tag{27}$$

with $V_h \subset \tilde{V}$, $Q_h \subset \tilde{Q}$, $W_h \subset \tilde{W}$.

In this definition, we indicate with Q_p^d the space of the polynomials of degree p and dimension d .

To ensure that the LBB condition is satisfied, we require the polynomial degree for the velocity to be higher than that of the pressure. In general, the polynomial degree used for the fluid is independent of the one used in the solid.

The Galerkin formulation is obtained by restricting (26) to V_h, Q_h, W_h . It reads:

$$\begin{aligned} & \text{Find } \mathbf{u}_h \in V_h, p_h \in Q_h, \mathbf{d}_h \in W_h \text{ such that} \\ & a_f(\mathbf{u}_h, \mathbf{v}_h) + b^T(p_h, \mathbf{v}_h) + b(\mathbf{u}_h, q_h) + a_s(\mathbf{d}_h, \mathbf{w}_h) \\ & \quad + c_u(\mathbf{u}_h, \mathbf{w}_h) + c_p(p_h, \mathbf{w}_h) = G(R\mathbf{u}_0, \mathbf{v}_h) \\ & \text{for all } \mathbf{v}_h \in V_h, q_h \in Q_h, \mathbf{w}_h \in W_h. \end{aligned} \quad (28)$$

Let $\dim(V_h) = N_h^u$, $\dim(Q_h) = N_h^p$, $\dim(W_h) = N_h^d$ be the cardinalities of the spaces.

Let us now define the bases for the spaces:

$$\begin{aligned} & \{\phi(\mathbf{x})_i\}_{i=1}^{N_h^u} \text{ basis for } V_h, \text{ such that } V_h = \text{span} \{\phi(\mathbf{x})_i\}_{i=1}^{N_h^u} \\ & \{\psi(\mathbf{x})_i\}_{i=1}^{N_h^p} \text{ basis for } Q_h, \text{ such that } Q_h = \text{span} \{\psi(\mathbf{x})_i\}_{i=1}^{N_h^p} \\ & \{\chi(\mathbf{x})_i\}_{i=1}^{N_h^d} \text{ basis for } W_h, \text{ such that } W_h = \text{span} \{\chi(\mathbf{x})_i\}_{i=1}^{N_h^d} \end{aligned} \quad (29)$$

where ϕ_i and χ_i are the vector-valued basis functions. We construct them from the scalar basis functions ϕ_j and χ_k (with $j = 1, 2, \dots, N_h^u/2$ and $k = 1, 2, \dots, N_h^d/2$):

$$\phi_1 = \begin{bmatrix} \phi_1 \\ 0 \end{bmatrix} \quad \phi_2 = \begin{bmatrix} 0 \\ \phi_1 \end{bmatrix} \quad \phi_3 = \begin{bmatrix} \phi_2 \\ 0 \end{bmatrix} \quad \dots \quad (30)$$

$$\chi_1 = \begin{bmatrix} \chi_1 \\ 0 \end{bmatrix} \quad \chi_2 = \begin{bmatrix} 0 \\ \chi_1 \end{bmatrix} \quad \chi_3 = \begin{bmatrix} \chi_2 \\ 0 \end{bmatrix} \quad \dots \quad (31)$$

with $\{\phi_i\}$, $\{\psi_i\}$ and $\{\chi_i\}$ Lagrangian bases of the spaces.

We can now express the elements of the spaces as a linear combination of the basis vectors:

$$\begin{aligned} \mathbf{u}_h(\mathbf{x}) &= \sum_{j=1}^{N_h^u} U_j \phi_j(\mathbf{x}) \quad \forall \mathbf{u}_h(\mathbf{x}) \in V_h \\ p_h(\mathbf{x}) &= \sum_{j=1}^{N_h^p} P_j \psi_j(\mathbf{x}) \quad \forall p_h(\mathbf{x}) \in Q_h \\ \mathbf{d}_h(\mathbf{x}) &= \sum_{j=1}^{N_h^d} D_j \chi_j(\mathbf{x}) \quad \forall \mathbf{d}_h(\mathbf{x}) \in W_h \end{aligned} \quad (32)$$

where $U_j, P_j, D_j \in \mathbb{R}$ are unknown.

As we said, we want to solve the problem monolithically; we have already defined (25) and (27), the expansions of the domains on all Ω .

We also want to define the solution as a monolithic element of Ω . For this aim, we define $Z \equiv \{\phi : \phi(\mathbf{x}) = 0\}$ as the set of the functions that are constantly equal to zero. Its dimension is zero and its discretization is $Z_h = Z$.

Our discrete vector-valued solution is

$$\begin{aligned} & \mathbf{y}_h \in Y_h \text{ such that} \\ Y_h = \{ \mathbf{y}_h = \{ \mathbf{v}_h, q_h, \mathbf{w}_h \} : \mathbf{y}_h|_{\Omega_f} \in Q_{p+1}^2 \times Q_p \times Z^2, \mathbf{y}_h|_{\Omega_s} \in Z^2 \times Z \times Q_r^2 \} \end{aligned} \quad (33)$$

We can build a basis $\{\Psi_j\}_{j=1}^{N_{el}}$ of Y_h from the bases defined in (29), with $N_{el} = N_h^u + N_h^p + N_h^d$. The elements of this basis can be defined as:

$$\Psi_1 = \begin{bmatrix} \phi_1 \\ 0 \\ 0 \end{bmatrix} \quad \Psi_2 = \begin{bmatrix} 0 \\ \psi_1 \\ 0 \end{bmatrix} \quad \Psi_3 = \begin{bmatrix} 0 \\ 0 \\ \chi_1 \end{bmatrix} \quad \Psi_4 = \begin{bmatrix} \phi_2 \\ 0 \\ 0 \end{bmatrix} \quad \dots \quad (34)$$

Since any generic $\mathbf{v}_h, q_h$ and $\mathbf{w}_h$ can be written as a linear combination of the elements of the bases of the spaces they belong to, a generic $\mathbf{y}_h \in Y_h$ can be written as:

$$\mathbf{y}_h = \begin{bmatrix} \mathbf{v}_h \\ q_h \\ \mathbf{w}_h \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^{N_h^u} V_j \phi_j \\ \sum_{j=1}^{N_h^p} Q_j \psi_j \\ \sum_{j=1}^{N_h^d} W_j \chi_j \end{bmatrix} \quad (35)$$

We now have a discrete space over all Ω , that is the space where we can find the monolithic solution to our discrete problem. In this space, the Galerkin formulation is:

$$\begin{aligned} & \text{Find } \mathbf{y}_h \in Y_h \text{ such that} \\ & \alpha(\mathbf{y}_h, \mathbf{x}_h) = G(R\mathbf{u}_0, \mathbf{x}_h) \\ & \text{for all } \mathbf{x}_h \in Y_h. \end{aligned} \quad (36)$$

where $\alpha(\mathbf{y}_h, \mathbf{x}_h)$ is equal to the sum of the bilinear forms $a_f(\mathbf{u}_h, \mathbf{v}_h) + b^T(p_h, \mathbf{v}_h) + b(\mathbf{u}_h, q_h) + a_s(\mathbf{d}_h, \mathbf{w}_h) + c_u(\mathbf{u}_h, \mathbf{w}_h) + c_p(p_h, \mathbf{w}_h)$, but it is defined on a generic test function $x_h \in Y_h$ and not for the test functions in the subspaces V_h, Q_h, W_h ; the function $G(R\mathbf{u}_0, \mathbf{x}_h)$ is equal to $G(R\mathbf{u}_0, \mathbf{v}_h)$ for the first N_h^u elements and it is equal to zero for the following ones.

2.3 Linear system

Instead of requiring that (36) holds for all functions $\mathbf{x}_h \in Y_h$, we can equivalently require it for each basis function Ψ_i , for $i = 1, 2, \dots, N_{el}$.

Then, the discrete weak formulation becomes: find $Y_j \in \mathbb{R}$, for $j = 1, 2, \dots, N_{el}$ such that

$$\sum_{j=1}^{N_{el}} Y_j \alpha(\Psi_j, \Psi_i) = G(R\mathbf{u}_0, \Psi_i) \quad \text{for } i = 1, 2, \dots, N_{el} \quad (37)$$

Equations in (37) can be rewritten as a linear system

$$M\mathbf{y} = \mathbf{g}, \quad (38)$$

where

$$\begin{aligned} \mathbf{y} &\in \mathbb{R}^{N_{el}}, \quad \mathbf{y} = (Y_1, Y_2, \dots, Y_{N_{el}})^T \\ M &\in \mathbb{R}^{N_{el} \times N_{el}}, \\ M_{ij} &= \begin{cases} a_f(\phi_j, \phi_i) & i, j \in \{1, \dots, N_h^u\} \\ b(\psi_j, \phi_i) & i \in \{1, \dots, N_h^u\}, j \in \{N_h^u, \dots, N_h^u + N_h^p\} \\ b^T(\phi_j, \psi_i) & i \in \{N_h^u, \dots, N_h^u + N_h^p\}, j \in \{1, \dots, N_h^u\} \\ a_s(\chi_j, \chi_i) & i, j \in \{N_h^u + N_h^p, \dots, N_h^u + N_h^p + N_h^d\} \\ c_u(\phi_j, \chi_i) & i \in \{N_h^u + N_h^p, \dots, N_h^u + N_h^p + N_h^d\}, j \in \{1, \dots, N_h^u\} \\ c_p(\psi_j, \chi_i) & i \in \{N_h^u + N_h^p, \dots, N_h^u + N_h^p + N_h^d\}, j \in \{N_h^u, \dots, N_h^u + N_h^p\} \end{cases} \\ \mathbf{g} &\in \mathbb{R}^{N_{el}}, \quad \mathbf{g}_i = G(R\mathbf{u}_0, \Psi_i) \end{aligned} \quad (39)$$

We can also write the linear system (38) as

$$\begin{bmatrix} A_{\text{vel}} & B^T & 0 \\ B & 0 & 0 \\ C_u & C_p & A_{\text{solid}} \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{P} \\ \mathbf{D} \end{bmatrix} = \begin{bmatrix} \mathbf{G} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} \quad (40)$$

3 Code implementation details

3.1 Use of hp-namespace for multi-physics coupling

As derived in the mathematical formulation, solving the problem with a monolithic approach requires the fluid velocity and pressure (defined in Ω_f) and the solid displacement (defined in Ω_s) to be handled within a unified global DoF handler. In deal.II, this is achieved by utilizing the classes and facilities of the hp namespace. While its original purpose is to vary the polynomial degree for numerical accuracy (*hp*-FEM), here it is exploited to assign structurally different finite elements to different cells, effectively managing the multi-physics coupling.

Specifically, a composite finite element is assigned to each cell based on its subdomain:

- Fluid cells use a system of elements $[Q_{p+1}]^{\text{dim}} \times Q_p \times [\text{FE_Nothing}]^{\text{dim}}$ for velocity, pressure, and displacement respectively, where p is the polynomial degree chosen for the pressure space (satisfying the LBB condition).

- Solid cells use $[\text{FE_Nothing}]^{\text{dim}} \times \text{FE_Nothing} \times [Q_r]^{\text{dim}}$, where r is the polynomial degree for the elements in Ω_s .

The `FE_Nothing` element strategically nullifies the degrees of freedom for specific variables where they are not physically present.

This allows the `hp::DoFHandler` to manage a single, monolithic system matrix where each row and column retains a clear physical meaning only in the appropriate subdomain. Consequently, the discrete global space Y_h is implemented using a `hp::FECollection`. The integration is performed with an appropriate `hp::QCollection` via an `hp::FEValues` object, with the `DoFHandler` operating strictly in hp-mode.

3.2 The classes

The implementation includes two template classes: `StokesBoundaryValues` and `FluidStructureProblem`, which are generic with respect to the problem dimensions (2D, 3D).

3.2.1 StokesBoundaryValues class

This class defines the Dirichlet boundary values for the Stokes problem; in particular, it represents a vector-valued function with $2 \cdot \text{dim} + 1$ components because we are working in the previously defined space Y_h .

The class exposes two methods: the first one, `double value()`, returns a specific component of the vector-valued function in a specific point of the domain; the other one, `void vector_value()`, fills a given vector with the values of the vector-valued function at a specific point.

3.2.2 FluidStructureProblem class

This is the primary class of the program, which builds and solves the problem in a hard-coded domain, given the following parameters:

- The polynomial degrees for the Stokes and the elasticity sub-problems.
- The fluid viscosity (ν).
- The parameters for the solid (λ and μ).

Mesh creation The `void make_grid()` method constructs a mesh of a hypercube domain with 8 elements per dimension, then it sets the `boundary_ID` of the faces located on the Dirichlet boundary of the Stokes problem.

Using a 8x8 initial mesh ensures that the interface Σ is always represented by a set of cell faces and never cut through the interior of an element.

Finally, it assigns the appropriate `material_id` to each cell based on the subdomain it belongs to.

Following this assignment, the `void set_active_fe_indices()` method determines the correct finite element to use on each locally owned cell by checking its subdomain and setting its active FE index accordingly.

Mesh refinement The `float refine_mesh(float tol)` method refines the mesh by first computing the error estimates for both the fluid and solid domains via the `KellyErrorEstimator<dim>::estimate()` method. It then normalizes these error estimates by their global norms across all MPI processes using `MPI_Allreduce` and scales them to balance the refinement between the two subdomains. Since the `KellyErrorEstimator` class cannot correctly evaluate the cells situated at the interface Σ , because adjacent cells belong to completely different physical domains (fluid velocity/pressure versus solid displacement), the method explicitly sets the error indicators to zero for those cells before executing the refinement.

Furthermore, dynamically generated child cells inherit `material_id` from their parent cells during mesh refinement. This exact inheritance ensures that the physical boundaries of the fluid and solid subdomains remain strictly defined, keeping the global domain mathematically consistent throughout all refinement cycles.

Ultimately, the method returns the computed global error norm.

We can see 9 cycles of mesh refinement in Figure 3.

Degrees of freedom setup The `void setup_dofs()` method extracts locally owned and locally relevant DoFs for the parallel architecture, and creates a sparsity pattern that also encodes the problem constraints.

First, it processes the constraints resulting from the hanging nodes, which are nodes located at the interface between two finite elements with different refinement levels (as shown in Figure 4), imposing continuity across these interfaces.

Next, it computes the constraints associated with the Dirichlet boundary conditions for both domains; specifically, it uses the `StokesBoundaryValues` class for the fluid and the `ZeroFunction` for the solid, enforcing the zero-displacement assumption on the solid's outer boundary. Then it adds the constraint ensuring zero fluid velocity at the fluid-solid interface Σ . This is achieved by iterating over the fluid cells: for each face, it checks if the neighboring cell has children. If not, the face lies on Σ if the neighbor belongs to the solid subdomain; if it has children, the algorithm verifies each child to see if its neighbor belongs to the solid subdomain, identifying the faces on Σ accordingly.

Finally, the method constructs the distributed sparsity pattern using `TrilinosWrappers::BlockSparsityPattern` combined with `DoFTools::make_flux_sparsity_pattern`, explicitly appending the cross-domain coupling entries across the solid-fluid faces to ensure correct global memory allocation across all MPI ranks.

System assembly The `void assemble_system()` method assembles the system matrices and the right-hand side to solve the problem. To construct them, it creates a quadrature formula for the Stokes problem and another for the elasticity problem. Then, using these quadratures and the original finite elements, it sets up the `hp::FEValues` object, as well as `FEFaceValues` and

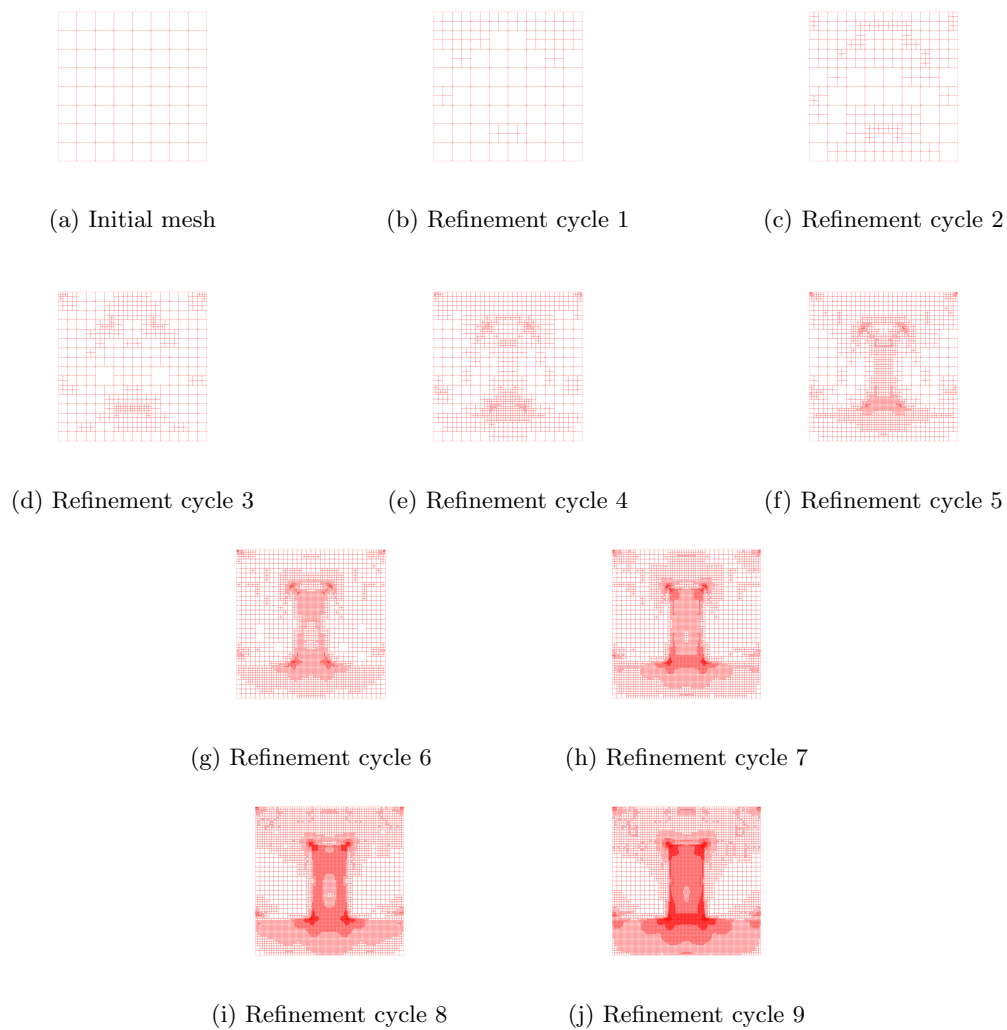


Figure 3: Mesh refinement cycles

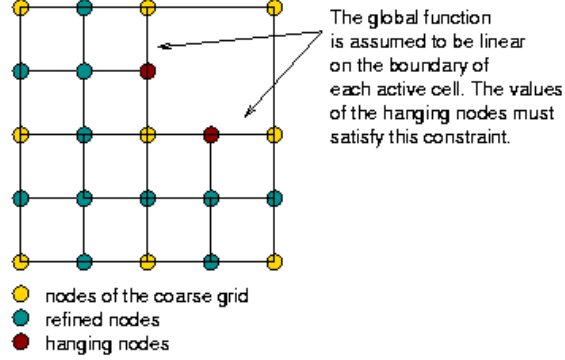


Figure 4: hanging nodes

`FESubFaceValues` for both sub-problems.

The `FEFaceValues` object is used to compute boundary integrals when the entire face of the current cell must be considered. Specifically, this occurs when computing the integral between two cells with the same refinement level, or when the current cell is more refined than its neighbor. Conversely, `FESubFaceValues` is used when only a portion of the face is considered, specifically when the current cell is coarser than its neighbor.

After setting up these auxiliary objects, the method iterates over the locally owned mesh cells, computing the local matrices according to the analytical formulation previously discussed, and then distributes them into the global matrix.

Finally, it constructs the interface matrix by iterating over the solid domain cells to identify those located on the interface Σ . Here, three cases can occur:

1. The current cell and its neighbor share the same refinement level and have no children: `FEFaceValues` is used for both sub-problems.
2. The current cell and its neighbor have the same refinement level, but the neighbor has children: the algorithm iterates over every child, using `FESubFaceValues` for the current (solid) cell and `FEFaceValues` for the neighbor (fluid) cell.
3. The neighbor cell is coarser than the current one: `FEFaceValues` is used for the current (solid) cell and `FESubFaceValues` for the neighbor (fluid) cell.

For each interface cell, after determining the appropriate evaluation objects, it calls the `void assemble_interface_term()` function. This function takes the chosen objects as arguments, assembles the local interface matrix based on the analytical model, and distributes it to the global matrix.

The system matrix is built in blocks. The upper-left block, $A_{\text{fluid}} \in \mathcal{M}_{(N_h^u + N_h^p) \times (N_h^u + N_h^p)}(\mathbb{R})$, represents the Stokes sub-problem. The lower-left block, $B \in \mathcal{M}_{(N_h^u + N_h^p) \times (N_h^d)}(\mathbb{R})$, represents the interactions at the interface, and the lower-right block is $A_{\text{solid}} \in \mathcal{M}_{N_h^d \times N_h^d}(\mathbb{R})$. The upper-right block is the zero matrix. This block-structured composition allows us to solve the linear

system iteratively using an ad hoc preconditioner, which is discussed in the following section.

To guarantee consistency in the parallel environment, `.compress(VectorOperation::add)` is systematically invoked on all distributed matrices and vectors to accumulate the independently computed ghost cell contributions.

Solution The assembled system is solved by the `void solve()` method, which uses the GMRES method to approximate the solution. Before applying this method, the system is preconditioned using the custom `FSIPreconditioner` class. This class computes the preconditioners for the two sub-problems independently and merges them together to construct the global preconditioner. Finally, it writes the solution to the locally owned `solution_owned` vector, and then correctly distributes it to the ghosted `solution` vector, so that the `void refine_mesh()` method will use it to compute the error estimation.

Problem run The `void run()` method encloses the problem setup and the solution-mesh-refinement cycle. It continues this cycle until the norm of the estimated error vector falls below a user-defined tolerance, or the iteration count reaches a predefined limit.

4 Preconditioner

4.1 Block-Triangular Approximation

As we derived in (40), the global matrix of the system we are studying can be explicitly written as:

$$M = \begin{pmatrix} A_{\text{vel}} & B^T & 0 \\ B & 0 & 0 \\ C_u & C_p & A_{\text{solid}} \end{pmatrix}$$

where the upper-left 2×2 submatrix represents the Stokes problem for fluid velocity and pressure, A_{solid} results from the elasticity equations for the displacements, and the bottom row components (C_u and C_p) come from the interface conditions.

Note that the (1,1) pressure block in the system matrix is zero, as no pressure-pressure coupling appears in the Stokes formulation. However, a well-known result in the preconditioning of saddle-point problems is that the pressure block of the Schur complement is well approximated by the scaled pressure mass matrix M_p , defined as:

$$M_p = \frac{1}{\nu} \int_{\Omega_f} p_h q_h dx \quad (41)$$

This matrix is assembled separately in the `mass_matrix` object and is used in place of the zero (1,1) block when applying the preconditioner. It is symmetric

and positive definite, which justifies the use of a CG solver for this block (see Section 4.2.2). For the coupled one-way system, the exact block-triangular inverse and its approximation via sub-preconditioners follow the same structure described previously, with $\tilde{M}_p^{-1}$ replacing the (1,1) block in the preconditioner.

For a generic coupled system with a one-way coupling representation, the matrix and its inverse take the form:

$$A_{\text{global}} = \begin{pmatrix} A_{\text{fluid}} & 0 \\ C & A_{\text{solid}} \end{pmatrix}, \quad A_{\text{global}}^{-1} = \begin{pmatrix} A_{\text{fluid}}^{-1} & 0 \\ -A_{\text{solid}}^{-1} C A_{\text{fluid}}^{-1} & A_{\text{solid}}^{-1} \end{pmatrix}$$

Applying this exact inverse matrix requires only one solve with A_{fluid} and A_{solid} each. We can therefore expect that a good preconditioner can be formed by approximating the inverse blocks with separate sub-preconditioners:

$$A_{\text{global}}^{-1} \approx \begin{pmatrix} \tilde{A}_{\text{fluid}}^{-1} & 0 \\ -\tilde{A}_{\text{solid}}^{-1} C \tilde{A}_{\text{fluid}}^{-1} & \tilde{A}_{\text{solid}}^{-1} \end{pmatrix}$$

This means we only need good preconditioners for the individual Stokes and elasticity equations separately. In our 3×3 formulation, this translates to finding robust preconditioners for the velocity stiffness ($\tilde{A}_{\text{vel}}^{-1}$), the pressure mass matrix ($\tilde{M}_p^{-1}$), and the solid stiffness ($\tilde{A}_{\text{solid}}^{-1}$). When combining these into a multiphysics preconditioner, there is a strict balancing act between the number of iterations to convergence and the cost of applying the preconditioner on a per-iteration basis. It is an additional constraint that the sub-preconditioners must converge at roughly the same rate; otherwise, the cheap one may drive up the global number of iterations while the expensive one drives up the cost-per-iteration.

4.2 Implementation details (FSIPreconditioner)

The custom `FSIPreconditioner` class implements this lower block-triangular matrix inversion via forward substitution in its `vmult` method. The application is divided into three sequential steps, targeting the 3×3 block structure (Velocity, Pressure, Displacement):

4.2.1 Fluid Velocity (Block 0)

The fluid velocity block is approximated using an iterative GMRES solver. The preconditioner chosen for this step is an Algebraic Multigrid (AMG) method (`TrilinosWrappers::PreconditionAMG`). It is specifically configured for elliptic problems with higher-order elements and uses 2 smoother sweeps to efficiently reduce high-frequency error components.

4.2.2 Fluid Pressure (Block 1)

To solve for the pressure block, the algorithm first updates the right-hand side using the newly computed velocity (`tmp_p = src_1 - B * dst_0`). The pressure mass matrix is then inverted using a Conjugate Gradient (CG) solver preconditioned with an Incomplete LU factorization (ILU).

4.2.3 Solid Displacement (Block 2)

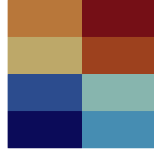
Finally, the right-hand side for the solid domain is updated using both the computed fluid velocity and pressure (`tmp_d = src_2 - Cu * dst_0 - Cp * dst_1`). The displacement stiffness matrix is inverted using a GMRES solver. Similar to the velocity block, an AMG preconditioner is used here, but it uses 3 smoother sweeps to guarantee robustness.

5 Parallelization

We parallelized the deal.II step-46 tutorial, that was our starting point for the implementation of this project.

5.1 Mesh Partitioning and Refinement

To enable distributed computing, the standard `Triangulation<dim>` is replaced with `parallel::distributed::Triangulation<dim>`, which automatically handles mesh partitioning across MPI processes. For adaptive mesh refinement, the `maximum_smoothing` flag is not used. This flag requires all children of a parent cell to be at the same level, which is violated by the `refine_and_coarsen_fixed_number` method because it refines 30% of cells. Instead, the `limit_level_difference_at_vertices` flag is applied to enforce a standard 2:1 balance condition, ensuring that neighboring cells differ by at most one refinement level to keep hanging node constraints manageable. Parallel mesh refinement is executed via `parallel::distributed::GridRefinement::refine_and_coarsen_fixed_number`. Before scaling the per-cell error vectors, a global reduction via `MPI_Allreduce` is performed to compute the global error norm. During iterations, the algorithm skips non-locally-owned cells by checking `!cell->is_locally_owned()`. Additionally, it skips artificial cells, which contain no valid data and act merely as placeholders to maintain consistent tree structures across processes, using `cell->is_artificial()`.



(a) Partitioning at step 0



(b) Partitioning at step 9

5.2 Distributed Data Structures and Degrees of Freedom

Data structures for vectors and matrices are replaced with their Trilinos parallel equivalents. The vectors for the solution and right-hand side use

`TrilinosWrappers::MPI::BlockVector`. Specifically, a writable `solution_owned` vector without ghost entries is maintained alongside a `solution` vector that includes ghost entries for output and error estimation. The system matrices and the FSI preconditioner components use `TrilinosWrappers::BlockSparseMatrix` and `TrilinosWrappers::SparseMatrix`. Managing the distribution of degrees of freedom requires tracking `locally_owned_dofs` and `locally_relevant_dofs`. These index sets are then used to extract per-block index sets via the `get_view(...)` method.

5.3 Sparsity Patterns and Matrix Assembly

The dynamic sparsity pattern is replaced by `TrilinosWrappers::BlockSparsityPattern`, initialized explicitly with the owned and relevant DoF sets.

Building the pattern requires `DoFTools::make_flux_sparsity_pattern` because standard sparsity pattern generation would entirely miss the solid-fluid coupling entries linking DoFs across different cells on a face. This function is called using

`numbers::invalid_subdomain_id` to build entries for all subdomains simultaneously, capturing all ghost contributions. The flag to retain constrained DoF entries is set to `true`, providing necessary diagonal entries for the linear solver.

Since cell coupling excludes pressure-pressure terms, these are added back subsequently to ensure the (1,1) block is populated. A dedicated sparsity pattern containing only pressure-pressure coupling is built for the pressure mass matrix.

During assembly, extracting strictly the rows and columns corresponding to pressure DoFs enables the correct distribution of the submatrix into the (1,1) block without misplacing zero entries into the velocity or displacement blocks. Following assembly, all matrices and vectors call `.compress(VectorOperation::add)`. This triggers a global MPI communication that sums the independently computed partial contributions from ghost cell boundaries into globally consistent values on the owning processes.

5.4 Solving, Output, and Post-processing

The linear solver targets the `solution_owned` vector directly. Upon completion, this data is assigned to the `solution` vector to correctly populate the ghosted entries.

Terminal output is strictly managed by wrapping `std::cout` in a `ConditionalOStream`, restricting print operations to MPI process rank 0.

To evaluate the parallel performance and scalability of the code, an execution timer has been introduced in the main routine using `std::chrono::high_resolution_clock`. This timer records the total wall time required for the problem setup and the entire solution cycle. At the end of the execution, the elapsed time is printed to the terminal exclusively by the master

process (checking
`dealii::Utilities::MPI::this_mpi_process(MPI_COMM_WORLD) == 0`). This tracking mechanism allows for an efficient comparison of the computational time when scaling the execution across different numbers of processors.

Finally, VTK output writing is updated to use
`data_out.write_vtu_with_pvtu_record(...)`.

6 Numerical results

6.1 Final results

In the Figures 6, 7 and 8 we can observe the output at the end of the 9th refinement step.

In the velocity plot (Figure 6), we can see the fluid entering from the top-left portion of the boundary Γ_{f1} , driven by the imposed Dirichlet condition. The flow travels downwards, turns right to bypass the solid obstacle, and finally exits through the top-right section of the domain. Furthermore, the velocity profile strictly adheres to the no-slip condition ($\mathbf{u} = 0$) along the fluid-solid interface Σ .

In the pressure plot (Figure 7), a distinct pressure gradient is visible, which acts as the primary driving force for the fluid. The left region exhibits high pressure (red area) as the fluid flow encounters the left wall of the solid structure. As the fluid passes over the obstacle and reaches the right side, the pressure drops significantly (blue area), consistently with the free-flow (homogeneous Neumann) condition imposed on Γ_{f2} .

In the displacement plot (Figure 8), we observe the mechanical response of the solid. The force per unit area exerted by the fluid's pressure and viscous stress on the interface directly causes the structural deformation. The displacement is strictly zero at the bottom boundary, reflecting the fixed condition enforced on Γ_s , and reaches its maximum amplitude at the top of the solid, where the fluid impact is most severe.

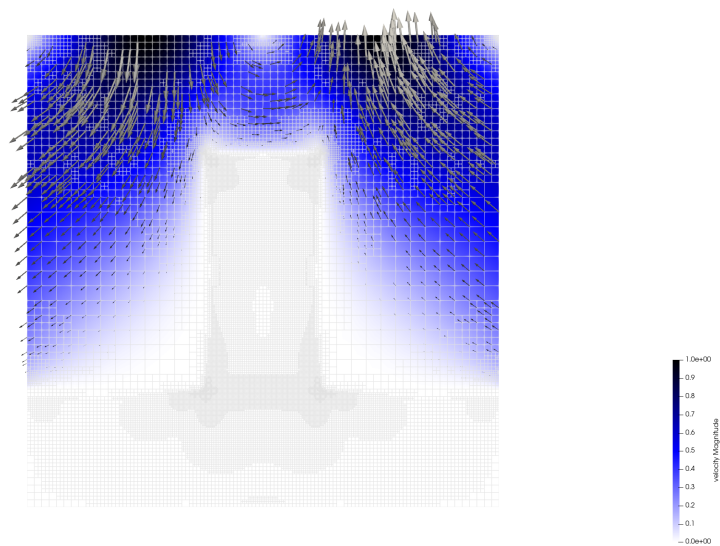


Figure 6: Final velocity

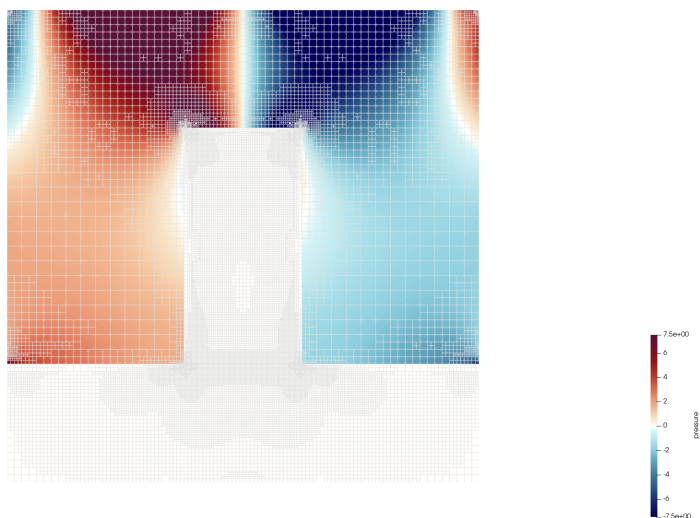


Figure 7: Final pressure

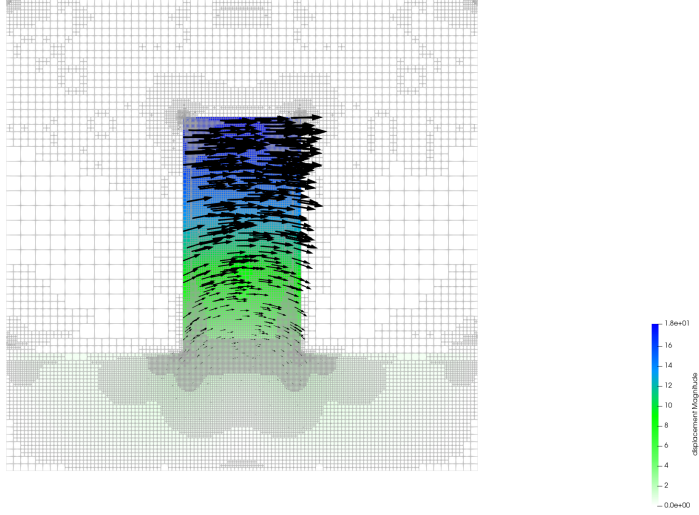


Figure 8: Final displacement

6.2 Scalability

6.2.1 Approach and setup

The scaling analysis was performed on the HPC cluster of the MOX Laboratory, whose CPU nodes are equipped with 56 physical cores each. With hyperthreading enabled, each node exposes 112 logical threads to the operating system. The job scheduler (OpenPBS) enforces a per-job limit of 28 logical threads.

An important aspect of the software stack is that deal.II is linked against Trilinos, which in turn is compiled with OpenMP support. This means that, by default, each MPI process may spawn multiple OpenMP threads for shared-memory parallelism within Trilinos' linear algebra routines (e.g. within the AMG preconditioner setup and application). On the cluster, the environment variable `OMP_NUM_THREADS` was set to 4 by default, meaning that with N MPI processes the total number of active threads would be $4N$, which can easily exceed the available physical cores.

To obtain clean and reproducible measurements, all scaling runs were performed with `OMP_NUM_THREADS=1`, passed explicitly to `mpirun` via the `-x` flag.

This ensures that each MPI process uses exactly one thread, making the execution purely MPI-parallel and the wall times directly comparable across different values of N .

A second critical choice concerns the mesh refinement strategy. The original code used adaptive mesh refinement driven by a Kelly error estimator. However, adaptive refinement in a distributed setting is non-deterministic with respect to the number of MPI processes: the mesh partitioning and the subsequent refinement pattern depend on how cells are distributed among processes, so different

values of N produce meshes with different numbers of degrees of freedom and different sparsity structures. This makes wall times fundamentally incomparable across runs, since the solver may converge in a different number of iterations for each configuration.

To address this, adaptive refinement was replaced by uniform global refinement (`triangulation.refine_global(1)`) at each cycle, and the convergence tolerance was set to a negative value to ensure that all cycles are always executed regardless of the estimated error. This guarantees that every run, independently of N , solves exactly the same sequence of linear systems on the same mesh hierarchy, making the timing results directly comparable and the scaling analysis meaningful.

6.2.2 Numerical results

Wall Time vs Number of Processors The wall time decreases monotonically with the number of processors for both problem sizes, confirming that the parallelisation is effective. However, the reduction is far from linear: doubling the number of processors does not halve the execution time, indicating significant parallel overhead. The 6-cycle case shows a steeper initial drop, suggesting that larger problems benefit more from additional processors in absolute terms. This can be seen in the first image in Figure (9).

Speedup vs Number of Processors Both curves lie well below the ideal linear speedup. The 5-cycle case (106k DoF) reaches a speedup of only 5.8 at $N=28$, while the 6-cycle case (421k DoF) achieves 9.6 at $N=28$ with fewer data points (second image in Figure (9)). This improvement with problem size is consistent with Amdahl’s law: as the problem grows, the parallel fraction dominates over the fixed serial overhead, pushing the speedup closer to the ideal. Nevertheless, both curves flatten significantly for large N , indicating that communication overhead becomes the bottleneck.

Efficiency vs Number of Processors The efficiency plot, in the third image in Figure (9), most clearly illustrates the impact of problem size. For 5 cycles, efficiency drops sharply from 100% at $N=1$ to roughly 21% at $N=28$, with most of the degradation occurring already between $N=1$ and $N=4$. For 6 cycles, efficiency at $N=4$ is 68% compared to 54% for 5 cycles — a substantial improvement. This trend strongly supports the conclusion that the problem size is the primary limiting factor: with only 3800 DoF per process at $N=28$ for the 5-cycle case, the ratio of communication to computation is unfavourable. Increasing the problem size to 421k DoF improves this ratio and leads to better parallel efficiency.

Efficiency vs DoF per Process This plot, shown in Figure (10), directly visualises the relationship between the local problem size — measured in degrees

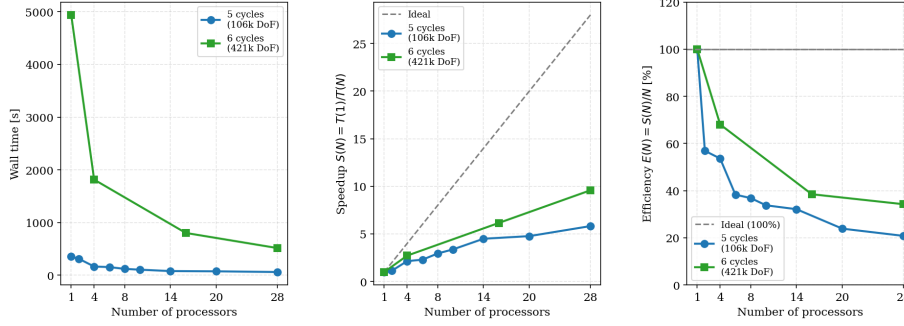


Figure 9: Strong scaling comparison

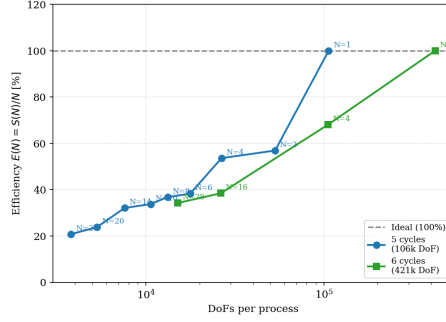


Figure 10: Efficiency vs DoFs per process

of freedom per process — and parallel efficiency. The horizontal axis is in logarithmic scale to better capture the wide range of values, from 3800 DoF/process ($N=28$, 5 cycles) to 421k DoF/process ($N=1$, 6 cycles).

The key observation is that efficiency increases consistently with the number of DoF per process, regardless of which cycle count the data comes from. Points from the 5-cycle and 6-cycle cases follow a similar trend, suggesting that DoF/process is the primary driver of efficiency rather than the absolute problem size or the number of processors alone.

For both cases, efficiency approaches 100% only when a single process handles the entire problem ($N=1$), and degrades as the workload per process decreases. Notably, the 6-cycle points at equivalent DoF/process values show slightly higher efficiency than the 5-cycle points, which may be attributed to the better ratio of computation to communication overhead in deeper refinement levels where local matrices are denser.

The plot also highlights a practical threshold: below approximately 10k–20k DoF per process, efficiency drops below 40% and continues to deteriorate rapidly. This suggests that for this FSI solver, a minimum of around 20k DoF per process is needed to achieve acceptable parallel efficiency above 50%.

7 3D

The three-dimensional extension of the problem highlights the capability of the monolithic solver to handle complex, multi-axial stress states.

In the velocity plot (Figure 11), the most prominent feature is the spatial distribution of the inlet velocity on the top boundary Γ_{f1} . Driven by the Dirichlet condition $\mathbf{u} = (0, 0, \sin(\pi x) \sin(\pi y))^T$, the flow exhibits a distinct pattern with localized high-velocity regions (the four lobes visible on the top face) rather than a uniform planar flow. This non-uniform inflow travels downwards, systematically enveloping the central solid structure.

Consequently, the pressure field (Figure 12) is significantly more intricate than its 2D counterpart. Instead of a simple planar gradient, we observe a complex 3D pressure distribution. The pressure strategically adapts to balance the highly localized velocity peaks from the inlet, generating the necessary gradients to force the incompressible fluid around the solid block in all three dimensions.

Finally, the displacement plot (Figure 13) demonstrates the resulting structural response. Because the hydrodynamic load is non-uniform, the solid block does not merely bend along a single plane; instead, it is subjected to a fully three-dimensional deformation, including potential torsion. The highest displacement values are concentrated asymmetrically on the upper edges and corners of the solid, precisely where the peak pressure and viscous shear stresses from the surrounding fluid are localized.

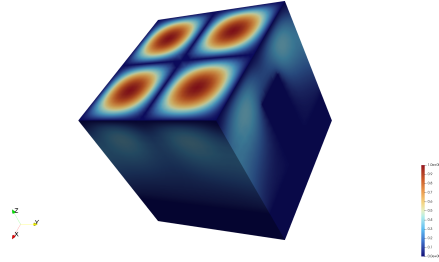


Figure 11: Velocity in 3D

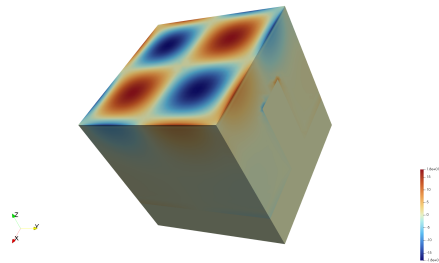


Figure 12: Pressure in 3D

